

# Script-based parametric modelling for personalised upper-limb prosthetics

## Introduction

Parametric modelling is commonly framed as a process where final geometry results from algorithms driven by input parameters rather than fixed, manual geometry edits. In the context of personalised upper-limb prosthetics, this framing aligns with treating a prosthesis as a configurable family of solutions that can be regenerated for different bodies, functional requirements, and manufacturing constraints. The literature reviewed here connects script-based approaches (with OpenSCAD as the most frequently discussed example) to goals central to Design for Additive Manufacturing (DfAM) and user-centred outcomes, including explicit control of parameters and dependencies, scalable variant generation, and the possibility of building accessible configuration interfaces on top of code-defined models.

At the same time, the sources also show that the benefits of script-based CAD are intertwined with limitations that matter for prosthetic design practice, including interoperability constraints (e.g., mesh-only export and loss of parametric information), cognitive burdens of programming and spatial transformations, and challenges validating print outcomes in the face of printer/material uncertainty. This report synthesises those contributions thematically, with attention to how parameters, constraints, anthropometric inputs, and printability/structural integrity strategies are technically structured in script-based versus graphical parametric environments.

## Conceptual foundations

Parametric approaches are described as introducing “changeability of the parameters into the ongoing design process,” and requiring designers to describe objects as rules of geometric dependencies and constraints that can be “translated” into a digital parametric model. In this view, parametric and generative methods are procedural—“based more on procedure than geometric form”—and are argued to potentially connect form generation more closely to how a design is built or performs. For a Research Through Design (RtD) dissertation, this supports treating the parametric model as a material research artefact: an executable procedure that encodes and externalises design knowledge, rather than only a geometry outcome.

A recurring conceptual theme is that parametric modelling enables efficient exploration of a “design space” via automated, scripted workflows, while parameterising geometry remains a non-trivial challenge. This challenge becomes more acute when parameters are coupled: the reviewed optimisation-oriented framing explicitly states that “parameters are not independent” and therefore involve relationships that must be maintained during changes. For personalised prosthetics, this coupling is not incidental: anthropometric fit, comfort, component clearances, and structural requirements are typically interdependent, and the “family of solutions” mindset requires designers to plan for those dependencies early in the modelling strategy.

Robustness and reusability are also conceptualised as key downstream requirements because parametric models are repurposed into manufacturing, engineering analysis, and optimisation activities. Methodologically, this connects to claims that achieving fully parameterised and adaptive products depends on “overall strategy, modelling methods, established procedures and approaches,” and that reusability in modelling methodologies has been “neglected,” motivating roadmaps that progress from standardisation to expert rules and automation. This line of work matters for prosthetics because a prosthetic “master model” that supports repeated patient-specific regeneration must remain modifiable and valid across many iterations and stakeholders, including makers, clinicians, and users.

## Comparison between code-based and graphical parametric tools

The literature distinguishes between text-based, programming-oriented CAD (e.g., OpenSCAD) and graphical/visual-programming environments (e.g., Grasshopper, Dynamo) that represent algorithms as node-and-wire dataflow networks. Grasshopper is explicitly described as a “visual programming language used mainly to generate geometries,” and visual programming is repeatedly motivated by enabling interactive parameter changes with continuous feedback that facilitates search for “optimal design solutions”. Complementary accounts emphasise that immediate visual feedback helps designers understand relationships as they manipulate user-defined parameters, which can support iterative exploration in early-stage design and analysis-driven loops.

Script-based CAD is framed differently: programmes are described as “precise specifications” that identify which elements should be modified across valid parameter combinations using mechanisms such as loops or semantic selection queries (including in OpenSCAD and other scriptable CAD systems). In open hardware contexts, the benefits of programming-language-based design are stated as enabling parametric design and providing “source code for the hardware,” which positions the design artefact closer to software and facilitates sharing and reproducibility. For personalised prosthetics, this code-as-specification framing is particularly relevant where distributed communities adapt and remake designs, since the parametric script can function as a compact, editable description of how to regenerate fit-specific parts.

However, comparative evidence also shows that tool choice influences late-stage flexibility. Product-platform work reports that Grasshopper can strongly support customising geometric parameters, but that initial parameter/constraint definitions may restrict modifications in later phases, and adaptability becomes more limited when customisation involves interdependent parameters related to ergonomics or usability. This suggests that both code-based and visual approaches can suffer from “lock-in” if dependency structures are poorly aligned with evolving design requirements, which is salient in prosthetic RtD where iterative user feedback can change what should be parameterised and constrained over time.

Hybrid approaches are presented as a mitigation direction. PotScript demonstrates integrating direct manipulation into parametric design so that complex forms can be created with small sets of user actions, using transformations that can be composed in a visual postfix language driven by manipulable curves. Other work reports that using GUI interactions to synthesise programmatic queries can improve speed and accuracy, and OpenSCAD-focused HCI research shows that bidirectional navigation and direct manipulation can be engineered so that view interactions update code coherently and help users define parametric properties by extracting information from the view.

The table below summarises differences that are repeatedly emphasised in the cited sources and that are directly relevant to prosthetic personalisation and DfAM workflows.

| Dimension                       | Script-based parametric CAD                                                                                                                                       | Graphical parametric CAD                                                                                                         |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Representation of design intent | Programmes act as precise specifications, using loops/queries to define modifiable elements across parameter combinations                                         | Visual programming language for generating geometries; relationships are manipulated through a node network with visual feedback |
| Interactive feedback            | OpenSCAD is described as not being an interactive modeler and not focusing on “artistic aspects,” foregrounding code-driven construction over direct manipulation | Interactive change of models and continuous feedback facilitate searching for optimal solutions                                  |

| Dimension                    | Script-based parametric CAD                                                                                                       | Graphical parametric CAD                                                                                                                                            |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Late-stage flexibility risks | OpenSCAD users report code-view navigation and debugging/validation difficulties that can slow iterative change and verification  | Initial parameter/constraint definitions can restrict later modifications; interdependent ergonomic/usability requirements can be hard to translate into algorithms |
| Interoperability             | OpenSCAD cannot export to STEP and exports tessellated formats; parametric information is lost once internal models are generated | Visual toolchains often integrate with analysis environments and plugins, enabling analysis in CAD loops (e.g., Rhino/Grasshopper-based workflows)                  |

## Technical characteristics of OpenSCAD

OpenSCAD is described as an open-source parametric constructive solid geometry (CSG) programming-based CAD application, and as not being an interactive modeler that focuses on artistic aspects of 3D modelling. Practitioner-oriented accounts characterise OpenSCAD as a script-based modelling technique (contrasted with graphical modelling applications), using CSG and extrusion of 2D outlines to create 3D models. This technical characterisation matters for prosthetics because upper-limb devices combine mechanical parts (where CSG primitives and boolean operations can be efficient) with anatomy-facing surfaces (where free-form curvature can be challenging to express in pure CSG without additional scaffolding or hybridisation).

A core OpenSCAD practice is defining parameters directly in the script file so that geometry can be adjusted by changing those parameters rather than editing geometry manually. A complementary empirical observation is that many OpenSCAD models use relatively simple algebraic expressions: an analysis of Thingiverse OpenSCAD models found geometric properties are mainly linear combinations of variables, with parameter categories dominated by primitives definitions and spatial transformations. For prosthetic personalisation, this supports a modelling tactic where anthropometric inputs primarily drive lengths, offsets, widths, and clearances through linear or near-linear dependency chains, while reserving more complex procedures for areas requiring non-linear fit or biomechanical behaviour.

In community practice, OpenSCAD parametric modelling is described as an iterative refactoring process: first create a target form, then identify code elements linked to adjustable design aspects and replace literals with variables specified when a module is called. This aligns with a modularisation strategy where reusable modules encapsulate subcomponents (e.g., finger segments, hinge blocks, strap interfaces) and expose a controlled parameter surface to reduce unintended consequences from changes.

Multiple sources also show interface and tooling layers designed to reduce OpenSCAD's coding burden. Bidirectional programming prototypes extend OpenSCAD so users can extract parametric expressions from the view for reuse in code, storing parametric definitions in CSG tree nodes and adding selectable handles around primitives to capture position/delta expressions. In a different direction, an intermediate Interactive Point-Cloud Modeling (IPCM) layer is introduced to enable concept-to-model transformation through 2D point-based representations, with functions consolidated into a pre-configured OpenSCAD template file so users call high-level functions (e.g., `create_polygon(points)`) rather than implementing low-level algorithms themselves. Web-based parameter

interfaces can also be generated by reading SCAD files and producing HTML controls, enabling novices to adjust parameters and then using OpenSCAD on a server to convert SCAD to STL.

## **Script-based parametric modelling for personalisation**

Personalised prosthetics are repeatedly framed as configurable product families whose geometry is regenerated from a parameter set rather than remodelled from scratch for each user. The need for such regeneration is linked to variability among individuals and to rehabilitation devices whose functional performance is tied to the individual's capabilities, motivating rapid customised design and manufacture of prostheses and orthoses.

### **Integration of anthropometric and scan-derived data**

The evidence shows two dominant input paradigms: measurement-driven templates and scan-to-model pipelines. In measurement-driven work, designers reduce the input space to a small set of “driver” dimensions and connect secondary dimensions to the main ones to keep geometry coherent under variation. One prosthetic hand study reports starting from eight parameters and finding only four important (grip circumference, palm breadth, thumb breadth, palm length), with all parameters connectable to a single driver (palm breadth), exemplifying an explicit dependency hierarchy for configurability. This kind of dependency structuring aligns with broader parametric design claims that parameters are coupled, implying that explicit relationships are necessary to maintain consistency during optimisation and variation.

Scan-to-model approaches populate parametric models using processed point-coordinate data, with “intelligent CAD models” being fed processed coordinates and automatically adapting geometry to match patient anatomy, and some pipelines using spreadsheets as the interface for patient-specific data input. For prosthetic sockets, semi-automatic workflows can involve large parameter sets derived from multiple stump measurements (e.g., 60 parameters from 23 measurements), reflecting the complexity of anatomically conforming interfaces. This contrasts with approaches that use only a few user parameters when baseline geometry is already patient-specific (e.g., displacement of a connection ring and thickness), underscoring that “parameter count” is partly a function of what is treated as an input geometry versus what is regenerated procedurally.

### **OpenSCAD-specific prosthetic examples**

Script-based customisation is explicitly documented in prosthetic contexts using OpenSCAD. A clinical case report describes parametric modelling of body-powered finger prostheses in OpenSCAD using nine stump-measurement parameters, followed by fused filament fabrication of individualised parts. Another OpenSCAD-related prosthetic workflow emphasises integrating CAD and CAM processes to facilitate bespoke prosthetic 3D printing, while noting OpenSCAD is required on the PC running the script to use the application as intended. Open-source community models also demonstrate parameter-to-STL scripts for fitment parts, such as a parametric gauntlet where users “adjust a bunch of parameters to create a custom made stl”.

### **User-centred configuration interfaces**

User-centred personalisation is also supported through configuration interfaces layered over parametric scripts. A free and open-source “3-D Customizer” approach reads SCAD files and generates HTML elements for parameter adjustment, with results showing novice users can adjust available parameters and save a customised file via a website, while OpenSCAD converts SCAD to STL on the server. In participatory design-oriented assistive-device work, involving patients in co-design teams is proposed as a way to directly transfer desires and creativity into

unconventional solutions, implying that a parametric system should be designed to support collaboration rather than only technician-driven configuration.

## **Implications for DfAM**

DfAM is framed as integrating manufacturability considerations—material selection, process parameters, support requirements, tolerances, and minimum feature limits—into the design workflow rather than deferring them to later stages. For prosthetics and orthotics, DfAM is repeatedly operationalised through staged workflows that separate design, validation, and manufacture (notably in socket development), supporting computational checks before committing to print iterations. Prosthesis-making is also described as starting from extraction of relevant anthropometric and biomechanics parameters and extending beyond fabrication to participatory and psychological considerations such as acceptance and training in daily use, which suggests that DfAM parameter schemas for prosthetics must incorporate both manufacturing constraints and situated use considerations.

### **Printability constraints as parameters**

For extrusion-based AM (FDM/FFF), the literature provides an explicit constraint inventory: a “final list of 54 major manufacturability constraints” intended to guide design decisions, while noting constraint relevance is context-dependent and tied to material choice and problem specifics. The same constraint-oriented work provides feature-scale heuristics (e.g., shells plus infill relative to nozzle size) that can be encoded as default bounds or validation checks inside a parametric generator to prevent unprintable thin features. From a script-based modelling perspective, this supports implementing printability checks as functions that reject or adjust parameter combinations that violate minimum feature sizes, wall thickness guidelines, or overhang-related heuristics.

### **Algorithmic printability evaluation and correction**

Computational printability methods formalise manufacturability prediction by simulating printing and partitioning layers into regions with local size measures, coupling printer resolution and material-specific minimum feature sizes to estimate deviations before manufacturing. Slice-level geometry correction approaches similarly construct a “meso-skeleton” within each slice to thicken regions smaller than the minimum printable size, with build-direction optimisation used to minimise required modifications, producing corrected models that can be printed “without any failures” given inputs of a surface mesh and a minimum feature size. These approaches demonstrate a DfAM logic that is natively algorithmic and therefore conceptually compatible with script-driven parametric pipelines, even when the geometric representation is mesh-based rather than B-rep.

### **Structural integrity and durability considerations**

Structural integrity strategies for AM prosthetics extend beyond static geometry rules. AM parts can be susceptible to cyclical loading failure and degradation, and points of integration (e.g., mechanical joints) may require added robustness, sometimes achieved by incorporating metal fixtures and optimising loading around stress concentrators. For socket and orthotic components, behaviour-driven strategies include varying infill type, external topology/shape, and material composition, and some work models infill elastic behaviour via numerical homogenisation and explicit infill geometry modelling, with finite element analyses showing 3D-printed components can meet stiffness requirements in prosthetic contexts. These contributions suggest that a personalised upper-limb prosthetic parametric model can support structural integrity by parameterising reinforcement zones, minimum radii, local thickness, and infill strategies, while situating those choices within validation stages that reflect the device’s loading scenarios and use conditions.

## Advantages for reproducibility, scalability, and automation

Script-based workflows are repeatedly positioned as mechanisms for turning geometry into an executable, repeatable process that can be iterated at scale rather than manually remodelled for each variant. Automation is defined as access to modelling or analysis functionalities via built-in programming languages, and reducing manual work by automating repetitive tasks using scripts is presented as a direct route to faster iteration and reduced human error.

Reproducibility and shareability are explicitly linked to code artefacts. Designing open-source hardware using a programming language is said to offer two benefits: enabling parametric design and providing source code for the hardware, and OpenSCAD is described as widely used in open-source hardware parametric design and open-source labware communities. For personalised prosthetics, where many designs are shared openly online, this suggests that script-based models can support more auditable and reproducible modification than distributing only mesh files, because the parameter definitions and generation logic remain accessible in the source code.

Scalable automation is illustrated through pipelines that accept structured input data and generate outputs rapidly. A fully automated pipeline built on a large-scale statistical shape model is reported to accept a single facial image and produce a wearable CAD model in under a minute, with benefits including a significant reduction in design time and complete removal of manual CAD modelling steps. While not prosthetics-specific, this demonstrates how parameter-to-geometry mappings can be orchestrated into end-to-end personalisation pipelines when inputs are well-structured and model regeneration is robust.

Robustness under high-volume regeneration is treated as a technical condition for automation at scale. In automated processes producing large numbers of models without human intervention, explicit-reference modelling methodologies are reported to yield faster regeneration times while maintaining comparable robustness for some parts, and explicit models can achieve significantly higher robustness for more complex parts in optimisation scenarios. Complementary work introduces the concept of “intrinsic variability” in modelling methodologies—variance in parametric model structure due to differences in interpretation and execution—which affects optimisation effectiveness and thereby reliability of automated generation pipelines. For prosthetic RtD, this implies that the success of a script-based generator is partly a modelling-methodology problem: how dependencies, references, and subtractions are structured affects whether a large range of anthropometric inputs produces valid, manufacturable geometry without frequent manual repair.

## Limitations and challenges

OpenSCAD-specific HCI research documents recurring user difficulties in tasks including 3D spatial understanding, validation and code debugging, creation of organic shapes, and code-view navigation. Users also report difficulty verifying outcomes of difference/intersect operations that remove volume, and request additional derived feedback such as bounding boxes and centre of mass to support verification. Participants further stress that OpenSCAD’s code editor lacks advanced code-assistance features such as autocompletion, increasing authoring and refactoring overhead compared with mature programming environments.

Toolchain and interoperability constraints are also emphasised. OpenSCAD is reported to be unable to export industry-standard CAD formats such as STEP and to export only tessellated 3D file formats, with parametric information being lost once the internal model is generated. This limitation is tied to OpenSCAD’s use of the CGAL constructive solid geometry library and the claim that CGAL works with polygonal meshes rather than parametric models. In prosthetic pipelines that depend on downstream CAD interoperability (e.g., for clinical documentation, metrology loops, or multi-material workflows), this mesh-centric representation can complicate auditability and integration, even if it remains adequate for STL-based fabrication pipelines.

Complexity and cognitive load arise in both code and visual paradigms. OpenSCAD is described as “extraordinarily easy to set up,” but its functional programming paradigm and scoping rules may be “cumbersome” for users not used to them. Visual/parametric design environments can also demand significant cognitive work: a protocol study reports more cognitive actions per minute in a parametric design environment due to extra algorithmic space, while a geometric modelling environment prompted more focus on product–user relations, suggesting a potential risk that algorithmic modelling can displace attention from user-centred considerations if not deliberately managed.

Finally, printing introduces uncertainties that users find difficult to manage in a code-centric workflow, including uncertainties from printers and material properties, difficulty identifying where to correct the code after physical inspection, and challenges validating designs before printing. In prosthetic contexts where durability and safe function are essential, this difficulty is compounded by broader evidence that many 3D-printed prosthetic studies are proof-of-concept and that evidence on user acceptance, functionality, and durability is lacking in peer-reviewed literature.

## Research gaps and opportunities

Multiple gaps emerge at the intersection of script-based parametric modelling, personalisation, and DfAM. First, there is an interoperability gap: OpenSCAD’s inability to export STEP, its tessellated export, and loss of parametric information are repeatedly presented as central constraints on downstream workflows that rely on standard parametric exchange and long-term editability. Comparative work therefore proposes future directions such as isolating designers from lower-level kernel structures and integrating scripted models into graphical interfaces so non-programmers can parametrise designs more easily.

Second, there is a usability and interaction-design gap. OpenSCAD user research frames identified challenges as a research opportunity for HCI and suggests that programming-based CAD could benefit from enriched interaction with derived information in the view, coherently connected with code and supporting edits based on that information. Prototype systems such as Programming In the Model (PIM) point toward multi-directional live scripting and script localization within the model as important interaction features, while also noting that additional design work is required for better user experience. These directions align with bidirectional OpenSCAD extensions that extract parametric expressions from the view and report reductions in errors and entry barriers by reducing the mathematical skills required in programming-based CAD.

Third, DfAM research highlights a gap between design intent and physical realisation, including reproducibility issues and fragmented digital workflows, and proposes metrology-informed feedback loops (e.g., scanning/XCT) to support high-fidelity digital twins for structural integrity and functional reliability. In prosthetics specifically, there is also a reported lack of specific technical descriptions and clinical evidence supporting 3D-printed prostheses, and durability and reinforcement (including socket reinforcement materials and shear-stress failure modes) are identified as underexplored topics requiring long-term assessment. These gaps imply that a script-based prosthetic generator should not only produce variants, but also encode traceable validation steps and reporting outputs that can support repeatable testing and longitudinal evaluation within RtD.

Fourth, methodological work cautions that human-centred design practices can have limitations and argues for critical perspectives and complementary approaches, implying that user-centred evaluation of parametric prosthetic tools should be designed with attention to inclusivity and broader value considerations rather than relying exclusively on iterative feedback cycles. This is consistent with prosthetics process accounts that include participatory methods and psychological acceptance as part of bespoke prosthesis creation, suggesting that parameter schemas and configuration interfaces should be developed as socio-technical artefacts that mediate collaboration among patients, clinicians, and makers.

## Conclusion

The reviewed literature characterises script-based parametric modelling as an approach where geometry is generated by algorithms and explicit parameters, supporting a procedural view of design that can encode dependencies, constraints, and reusable knowledge in executable form. Compared with graphical parametric environments that emphasise interactive visual feedback, code-based workflows offer strong affordances for reproducibility and automation—particularly in open-source and distributed fabrication contexts—but can suffer from interoperability constraints, coding/mathematical burdens, and challenges in debugging and validation that affect usability and user-centred iteration.

For a master’s dissertation on personalised upper-limb prosthetics within an RtD and DfAM framework, the synthesis suggests a design space of contributions that is simultaneously technical and methodological: (i) designing robust parameter hierarchies that couple anthropometric and functional requirements coherently, (ii) embedding printability and structural integrity constraints as first-class parameters and checks, and (iii) developing interaction layers (web configurators, bidirectional editing, enriched derived feedback) that make script-based models accessible to non-programmers while preserving the transparency of code as a specification. The evidence also underscores that clinical validation and long-term durability remain underdeveloped for 3D-printed prostheses, so parametric modelling research is likely to have the most impact when paired with validation protocols, traceable reporting, and participatory evaluation that connects regeneration logic to real-world use and safety outcomes.

## Source table

The table below consolidates key sources evidenced in the provided excerpts. Cells use citations to point to the exact quoted evidence supporting the stated contribution.

| Authors | Year | Title                                            | Source | Main contribution                                                                   | Relevance to parametric modelling and OpenSCAD             | Number of citations |
|---------|------|--------------------------------------------------|--------|-------------------------------------------------------------------------------------|------------------------------------------------------------|---------------------|
| —       | —    | Parametric modelling definition (Woodbury, 2010) | —      | Defines parametric modelling as algorithm- and parameter-driven geometry generation | Foundational definition used to frame parametric workflows | —                   |



| Authors | Year | Title                                                          | Source                                                                                                                                                                                                            | Main contribution                                                                                                              | Relevance to parametric modelling and OpenSCAD                                                          | Number of citations |
|---------|------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|---------------------|
| —       | —    | OpenSCAD vs FreeCAD Python comparison (configurable OSH model) | <a href="https://journals.plos.org/plosone/article/file?id=10.1371/journal.pone.0225795&amp;type=printable">https://journals.plos.org/plosone/article/file?id=10.1371/journal.pone.0225795&amp;type=printable</a> | States OpenSCAD is widely used for open-source labware parametric modelling and discusses tool comparison setup                | Highlights OpenSCAD benefits and limitations (e.g., STEP export constraints appear in related excerpt)  | —                   |
| —       | —    | OpenSCAD limitations on exports                                | <a href="https://dx.plos.org/10.1371/journal.pone.0225795">https://dx.plos.org/10.1371/journal.pone.0225795</a>                                                                                                   | Reports inability to export STEP and that only tessellated formats are exported; parametric information is lost                | Key interoperability limitation for script-based CAD pipelines that need standard parametric exchange   | —                   |
| —       | —    | Free 3-D Customizer                                            | <a href="https://www.mdpi.com/2411-9660/1/1/5">https://www.mdpi.com/2411-9660/1/1/5</a>                                                                                                                           | Web interface that reads SCAD files and generates HTML inputs; novices can adjust parameters; SCAD-to-STL conversion on server | Demonstrates user-centred parameter exposure for OpenSCAD scripts and distributed fabrication workflows | —                   |

| Authors | Year | Title                                 | Source                                                                                                                              | Main contribution                                                                                                               | Relevance to parametric modelling and OpenSCAD                                                                                     | Number of citations |
|---------|------|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| —       | —    | OpenSCAD user challenges study        | <a href="https://inria.hal.science/hal-04475132/file/gonzalez24.pdf">https://inria.hal.science/hal-04475132/file/gonzalez24.pdf</a> | Reports OpenSCAD user difficulties in spatial understanding, validation/debugging, organic shapes, and code-view navigation     | Provides empirical basis for usability limitations and suggests needs for enriched interaction/communication of stored information | —                   |
| —       | —    | Programming In the Model              | —                                                                                                                                   | Live interface with side-by-side model and script, real-time updating, on-demand dependency representations, operation preview  | Suggests interaction patterns that can mitigate code-view separation in programming-based CAD                                      | —                   |
| —       | —    | Prosthetic finger case using OpenSCAD | <a href="https://pmc.ncbi.nlm.nih.gov/articles/PMC9276309/">https://pmc.ncbi.nlm.nih.gov/articles/PMC9276309/</a>                   | Parametric modelling in OpenSCAD using nine stump-measurement parameters; 1-day measurement-to-production and low cost reported | Direct prosthetics example of OpenSCAD-based anthropometric parameterisation and FFF fabrication                                   | —                   |

| Authors | Year | Title                                             | Source | Main contribution                                                                                                         | Relevance to parametric modelling and OpenSCAD                                                          | Number of citations |
|---------|------|---------------------------------------------------|--------|---------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|---------------------|
| —       | —    | FDM/FFF manufacturability constraints compilation | —      | Provides list of 54 major manufacturability constraints and a rule-of-thumb feature scale bound                           | Supports encoding printability constraints as parameter bounds/checks in prosthetic configurators       | —                   |
| —       | —    | Printability map method                           | —      | Simulates printing to estimate deviations; couples resolution and minimum feature sizes for evaluation before manufacture | Illustrates algorithmic manufacturability evaluation compatible with scripted pipelines                 | —                   |
| —       | —    | Slice-level correction via meso-skeleton          | —      | Thickens regions below minimum printable size; optimises build direction; outputs corrected printable model               | Suggests computational correction approaches that can complement parametric generators for printability | —                   |